Text-to-SVG Generation via Fine-Tuned Language Models: A Multi-Machine Training and Ablation Study

Ivan Aristy
NYU Tandon School of Engineering
CS-GY 9223 / ECE-GY 7123: Deep Learning, Spring 2026
ia2194@nyu.edu

Abstract

We present a system for generating Scalable Vector Graphics (SVG) code from natural language prompts, developed for the DL Spring 2026 Kaggle competition. Our approach fine-tunes Qwen2.5-Coder-1.5B-Instruct using QLoRA adapters and iterative merge-and-retrain, achieving a public leaderboard score of 16.87/100. We conduct extensive ablation studies across 10 experiments on a held-out validation set, revealing that the system prompt is the single most critical inference component (35-point impact), followed by fine-tuning (7.3 points) and token generation budget (4 points). We additionally find that LoRA adapters slightly outperform full fine-tuning despite higher cross-entropy loss, and that minimal system prompts outperform verbose ones. Training was conducted across two machines: an NVIDIA RTX 2000 Ada (16GB) for QLoRA and an NVIDIA DGX Spark (128GB unified memory) for full fine-tuning. Code, model weights, and experiment logs are publicly available.

GitHub: https://github.com/ivanearisty/svg-gen
Model Weights: https://huggingface.co/kaleidoscopicwhether/svg-gen-weights

1 Introduction

Scalable Vector Graphics (SVG) is a resolution-independent image format defined by XML markup, widely used for icons, logos, and web graphics. Unlike raster formats, SVGs encode visual content as geometric primitives (paths, shapes, transforms) with explicit coordinate specifications. This structural nature makes SVG generation an interesting challenge for language models: the output must be both syntactically valid XML *and* visually faithful to a natural language description.

Our approach fine-tunes a small (1.5B parameter) code-specialized language model using QLoRA (Dettmers et al., 2023), with several key technical contributions:

1. **Iterative merge-and-retrain:** We merge trained LoRA adapters into base weights and apply fresh adapters, progressively building SVG generation capability. This yielded our largest single improvement (+1.4 Kaggle points).
2. **Dual-machine training infrastructure:** QLoRA on an RTX 2000 Ada (16GB VRAM) and full fine-tuning on a DGX Spark (128GB unified memory), enabling us to compare training paradigms.
3. **Comprehensive ablation study:** 10 controlled experiments quantifying the contribution of each system component, revealing that the system prompt alone accounts for 65% of system performance.
4. **System prompt sensitivity analysis:** Demonstrating an inverse relationship between prompt verbosity and generation quality for fine-tuned models.

We achieve a final public leaderboard score of 16.87/100.

2 Dataset

2.1 Competition Data

The provided training dataset contains 50,000 (prompt, SVG) pairs. Prompts are short natural language descriptions (mean ∼100 characters, ∼30 tokens) of icons and simple scenes. SVGs range from 91 to 15,937 characters (median 2,110), with 82% using a `viewBox` of `0 0 200 200`.

SVG structure: The vast majority of training samples are single-path icons — 49.4% contain a single complex `<path>` element, and 47.7% are multi-element compositions with paths. Only 2.9%

are basic shapes (rectangles, circles). Path elements use cubic Bézier curves (`C` commands with 6 coordinates each), making coordinate efficiency the primary challenge.

2.2 Preprocessing Pipeline

We apply four preprocessing steps to all training SVGs, ordered by impact:

1. **Coordinate rounding** (54% character reduction): Training SVGs contain excessive decimal precision (e.g., `93.30000305175781`). We round all floats to 1 decimal place (`93.3`), reducing the dataset from 126MB to 58MB with negligible visual impact.

2. **Default attribute stripping:** Remove attributes already at SVG defaults (`fill-opacity="1"`, `stroke="none"`, etc.) — 178,000 instances found.

3. **Comment removal:** Strip 3,475 XML comments.

4. **Whitespace collapse:** Normalize redundant spaces and blank lines.

After cleaning, we filter SVGs by token count: any sample exceeding 1,900 Qwen tokens is removed, ensuring every training example fits completely within the 2,048-token context window. This is critical since the model must observe the closing `</svg>` tag during training to learn when to stop generating. The filtered dataset retains 45,874 samples (91.7%).

2.3 External Data Experiment

The competition permits external datasets. We curated additional samples from SVGX-SFT-1M (Xing, 2024) and OmniSVG/MMSVG-Icon (Team, 2024), creating expanded datasets of 79,516 (kepler) and 149,516 (Thor) samples. However, as detailed in Section 4.4.1, external data consistently degraded competition performance due to distribution mismatch. We ultimately trained our final models on competition-only data, and our highest-scoring model (16.87) was trained on the 45k filtered dataset without any external samples.

2.4 Tokenization Challenge

A fundamental bottleneck in our approach is Qwen2.5-Coder's digit-level tokenization. The tokenizer splits every digit and punctuation mark into separate tokens:

```
SVG text:  M 156.3 42.8
Qwen:      [M][ ][1][5][6][.][3]
           [ ][4][2][.][8]
Count:     12 tokens
```

By contrast, GPT-2-style tokenizers encode multi-digit numbers as single tokens (156 $\rightarrow$ 1 token), achieving $\sim$5 tokens for the same input. This means Qwen uses 2–3$\times$ more tokens for SVG coordinates than necessary, consuming context window capacity on digit sequences rather than structural patterns.

For a typical SVG with 200 coordinate values, this wastes $\sim$600 tokens — the difference between fitting a complex shape and truncating it. We investigated alternative preprocessing (relative path commands + integer rounding, achieving 40% token savings with 0.973 SSIM vs. original) but used 1-decimal absolute coordinates for our final submission to match proven configurations.

3 Model Description

3.1 Base Model Selection

We use **Qwen2.5-Coder-1.5B-Instruct** (Hui et al., 2024) as our base model. Selection criteria:

- **Code specialization:** Pre-trained on code corpora including markup languages, giving it prior knowledge of XML/SVG syntax.

- **Size constraint:** Competition limit of 4B parameters. At 1.5B, the model fits comfortably in 16GB VRAM with 4-bit quantization.

- **Instruction tuning:** The Instruct variant supports the chat template format needed for system-prompt-guided generation.

We also evaluated 3B and 7B Qwen variants. The 3B model was used for full fine-tuning on the DGX Spark but ultimately underperformed the 1.5B model due to training instability on the Blackwell GPU architecture (FlashAttention incompatibility with SM 12.1).

3.2 Training Approach: QLoRA with Merge-and-Retrain

Our training strategy evolved through three rounds:

Round 1 — LoRA r=16 (Modal A100): Initial fine-tuning with rank-16 LoRA adapters on 45,874 competition samples for 3 epochs. This established our first baseline (Kaggle score: 15.47). Training used 4-bit NF4 quantization via Unsloth (Han and

Han, 2024), `paged_adamw_8bit` optimizer, learning rate 2×10^{-4} with cosine schedule.

Round 2 — Merge + LoRA r=32 (kepler RTX 2000 Ada): We merge the Round 1 adapter into the base weights, permanently baking the Round 1 knowledge into the model parameters. A fresh rank-32 LoRA adapter (2× trainable parameters: 37M vs. 18M) is then applied to the merged model. This was trained on 45,000 competition-only samples for 2 epochs. Score: **16.87** — our best result, representing a +1.4 point improvement from the merge-and-retrain technique alone.

Round 3 — Full Fine-Tune (Thor DGX Spark): We merged the Round 2 adapter and performed full fine-tuning (all 1.5B parameters trainable) on the DGX Spark's 128GB unified memory. This achieved the lowest cross-entropy loss (0.308 vs. LoRA's 0.34 floor) but scored only 16.26 on Kaggle — *worse* than the LoRA model. See Section 5.2.4 for analysis.

3.3 Inference Configuration

All inference uses greedy decoding (`do_sample=False`) with repetition penalty 1.1 and `max_new_tokens=1024`. The model runs in 4-bit quantized mode on the RTX 2000 Ada at approximately 0.05 samples/second (20 seconds per SVG).

A post-processing pipeline extracts SVGs from model output: (1) locate `<svg` tag with attributes, (2) attempt repair if XML is invalid (close unclosed quotes/tags, append `</svg>`), (3) fall back to a minimal circle SVG if extraction fails, (4) normalize width/height to 256×256.

4 Experimentation

4.1 Training Configurations

Table 1 summarizes all training runs across both machines.

4.2 Inference Parameter Sweep

We tested inference parameters on the best model (LoRA r=32, competition-only):

4.3 Dual-Machine Infrastructure

Training was distributed across two machines connected via local network:

- **kepler** (NVIDIA RTX 2000 Ada, 16GB VRAM): All QLoRA training and inference. Uses 4-bit NF4 quantization via Unsloth, `paged_adamw_8bit` optimizer, gradient checkpointing.
- **Thor** (NVIDIA DGX Spark, 128GB unified memory): Full fine-tuning with all parameters trainable in bf16. Required Docker containers with `-runtime=nvidia` due to a process isolation issue where bare-metal training processes were silently killed by the system's session management. AdamW optimizer (full precision, ∼35GB for 1.5B model: 5.7GB params + 23GB optimizer + 5.7GB gradients).

Model checkpoints were transferred via `rsync` over LAN. Thor was training-only; inference required kepler due to PyTorch `generate()` incompatibility with the Blackwell SM 12.1 architecture.

4.4 Failed Experiments

4.4.1 External Training Data

Adding SVGX-SFT-1M and OmniSVG data (76k mixed vs. 45k competition-only) degraded Kaggle score from 16.87 to 14.64 (−2.23 points). The external SVGs use different viewBox conventions (128×128 vs. 200×200) and different stylistic patterns. **Distribution match matters more than dataset volume.**

4.4.2 Code Generation Approach

We experimented with teaching the model to output Python code using a custom `SVGBuilder` API instead of raw SVG markup. The hypothesis was that Qwen2.5-*Coder* would be better at writing Python than predicting coordinate digit sequences. The approach produced valid code for 824/1000 prompts but scored only 12.26 on Kaggle — worse than the fallback baseline. The programmatic API naturally produces basic shapes (`rect()`, `circle()`) that cannot match the complex Bézier paths in ground truth SVGs.

4.4.3 Best-of-N with Heuristic Scoring

We generated multiple SVGs per prompt across 4 inference runs with different settings, then selected the "best" per prompt using heuristic scoring (valid XML, length, complete paths). This did not improve over single-run greedy decoding because the heuristic (longest valid SVG) does not correlate with SSIM.

Run	Machine	Method	Data	Epochs	LR	Loss	Kaggle
LoRA r=16	Modal A100	QLoRA 4-bit	46k comp	3	2×10^{-4}	0.35	15.47
LoRA r=32 (mixed)	kepler	QLoRA 4-bit	76k mixed	2	2×10^{-4}	0.47	14.64
LoRA r=32 (comp)	kepler	QLoRA 4-bit	45k comp	2	2×10^{-4}	0.35	**16.87**
Full FT 1.5B	Thor	Full bf16	46k comp	1.8	2×10^{-5}	0.37	—
Refined FT	Thor	Full bf16	46k comp	2	5×10^{-6}	0.31	16.26
Code-gen	Thor	Full bf16	46k code	2	2×10^{-5}	0.32	12.26

Table 1: Training configurations and results. "comp" = competition-only data, "mixed" = competition + external. The merge-and-retrain LoRA r=32 on competition data achieved the best Kaggle score despite not having the lowest training loss.

Configuration	Kaggle	Δ
Greedy, rep=1.1 (baseline)	**16.87**	—
Greedy, rep=1.05	16.51	−0.36
Greedy, rep=1.15	15.88	−0.99
Temp=0.05, rep=1.1	16.42	−0.45

Table 2: Inference parameter sweep on Kaggle. Greedy decoding with repetition penalty 1.1 is optimal; any deviation degrades performance.

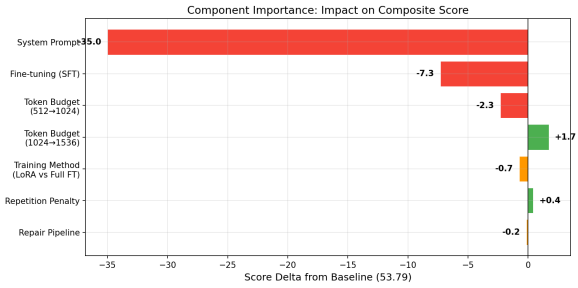

Figure 1: Component importance measured by score delta from baseline. The system prompt dominates all other components combined.

5 Results

5.1 Kaggle Competition Results

Table 3 shows all 9 Kaggle submissions. Our best public leaderboard score was 16.87.

Submission	Score	Notes
LoRA r=16, 3ep	15.47	First submission
r=32, 76k mixed	14.64	External data hurt
r=32, 45k comp	**16.87**	**Best public score**
temp=0.05	16.42	Sampling hurts
rep=1.05	16.51	Lower penalty hurts
rep=1.15	15.88	Higher penalty hurts
Best-of-N	15.72	Heuristic picking fails
Code-gen hybrid	12.26	Wrong approach
Refined FT	16.26	Full FT < LoRA

Table 3: Complete Kaggle submission history (public leaderboard).

5.2 Ablation Study

We conduct systematic ablation experiments on a held-out validation set of 200 samples stratified by SVG complexity from the training data. Each experiment removes or modifies exactly one component while holding all others constant. Scoring uses the same metric formula as the competition (Section 5.5).

5.2.1 System Prompt Is Everything (−35 points)

Removing the system prompt at inference is catastrophic: 192 out of 200 samples produce invalid SVGs (96% fallback rate), and the score drops from 53.79 to 18.80. The fine-tuned model learned to associate SVG generation with the presence of a system prompt during training. Without this signal, the model generates conversational text or code fragments instead of SVG markup.

5.2.2 Fine-Tuning Adds 7.3 Points

The base Qwen2.5-Coder-1.5B (no fine-tuning, with system prompt) scores 46.48 with 18 invalid outputs. Fine-tuning adds 7.31 points by teaching the model the specific SVG coordinate patterns and structural conventions of the training data. Notably, the base model still produces some valid SVGs (91% validity), demonstrating meaningful zero-shot SVG capability from code pretraining.

5.2.3 Token Budget: Linear Quality Scaling

Varying `max_new_tokens` from 512 to 1536 reveals an approximately linear relationship with quality (Figure 2). Each additional 512 tokens adds ∼2 points to the composite score. The improvement comes from allowing complex SVGs to complete rather than truncate mid-path. The 1536-token configuration achieves the highest score in our ablation study (55.50) with zero invalid outputs.

Ablation	What Changed	Score	Δ	Visual	Struct.	Invalid
Baseline	Full system	53.79	—	0.509	0.919	2
No system prompt	Remove system prompt	18.80	−34.99	0.156	0.886	192
No fine-tuning	Base Qwen2.5-Coder-1.5B	46.48	−7.31	0.447	0.888	18
max_tokens=512	Halve token budget	51.47	−2.32	0.491	0.885	4
Refined full FT	Full fine-tune (loss 0.308)	53.05	−0.74	0.502	0.911	0
No repair	Skip repair pipeline	53.63	−0.16	0.508	0.909	0
rep=1.0	No repetition penalty	54.21	+0.42	0.513	0.920	4
max_tokens=1536	1.5× token budget	**55.50**	+1.71	**0.525**	**0.933**	**0**

Table 4: Ablation study results on 200-sample validation set. Components ordered by negative impact magnitude. The system prompt is the most critical component; removing it causes 96% of outputs to fall back to the default SVG.

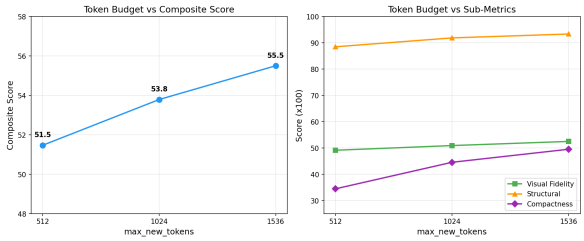

Figure 2: Token generation budget vs. quality. Left: composite score increases approximately linearly with budget. Right: all three sub-metrics improve monotonically.

System Prompt	Score	Δ
(none)	18.80	−34.99
"You generate SVG icons using path elements in a 200x200 viewBox..." (specific, 43 words)	51.66	−2.13
"You generate compact, valid SVG markup..." (original, 27 words)	53.79	—
"Output valid SVG code only." (minimal, 6 words)	**54.57**	+0.78

Table 5: System prompt sensitivity. Shorter prompts perform better: the 6-word minimal prompt outperforms the 27-word original and the 43-word specific prompt.

5.2.4 LoRA Outperforms Full Fine-Tuning

Despite achieving a lower cross-entropy loss (0.308 vs. 0.34), the fully fine-tuned model scores 53.05 vs. the LoRA baseline's 53.79 (−0.74). This pattern is consistent across both local scoring and Kaggle (16.26 vs. 16.87). We hypothesize that LoRA's constrained parameter space acts as an implicit regularizer, preventing the model from overfitting to training-set coordinate sequences while preserving the base model's general SVG understanding.

5.2.5 Negligible Components

The repetition penalty ($\Delta = +0.42$) and SVG repair pipeline ($\Delta = -0.16$) have minimal impact. The model already produces valid SVGs 98%+ of the time, making the repair pipeline largely redundant. The repetition penalty's effect is within noise for this task.

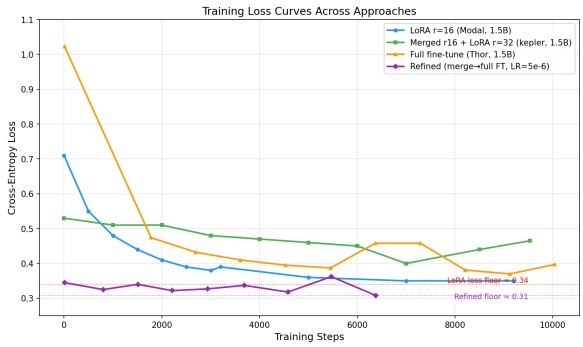

Figure 3: Training loss curves across four approaches. All methods converge to a loss floor of ∼0.30–0.40. The refined model (merge → full FT) achieves the lowest loss but not the best generation quality.

5.3 System Prompt Sensitivity Analysis

Given that the system prompt is the most impactful component, we investigate how prompt content affects generation quality.

We find an inverse relationship between prompt verbosity and generation quality. The minimal 6-word prompt ("Output valid SVG code only.") scores 0.78 points above the original, while the verbose prompt with specific structural instructions (mentioning viewBox, path elements, closing tags) scores 2.13 points below. Over-constraining the model with explicit instructions appears to interfere with patterns already learned during fine-tuning. The model knows SVG structure; the system prompt only needs to signal "produce SVG" without micromanaging how.

5.4 Training Loss Analysis

Figure 3 reveals a fundamental finding: **all training approaches converge to a cross-entropy loss floor of ∼0.30–0.40**, regardless of model size, training method, or dataset. The LoRA r=16 model plateaus at ∼0.34, the full fine-tune reaches ∼0.37, and the refined model (merge → full FT with low LR) pushes to 0.308. Yet the model with the *highest* loss (LoRA, 0.34) achieves the best Kaggle score.

This disconnect between CE loss and visual quality stems from three factors:

1. **One-to-many mapping:** Multiple valid SVGs exist per prompt. CE loss penalizes any deviation from the single reference, even if the generated SVG is visually equivalent.

2. **Digit-level loss:** Each digit is a separate CE term. Generating coordinates `157.1` instead of `156.3` incurs 4 wrong-digit penalties despite sub-pixel visual difference.

3. **Loss ≠ metric:** CE loss on tokens optimizes per-token accuracy, not SSIM/EdgeF1 on rendered images.

5.5 Scoring Details

For completeness, the competition scoring formula used for our local ablation evaluation:

$$V_i = 0.7 \cdot \mathrm{SSIM}(I_p, I_g) + 0.3 \cdot \mathrm{EdgeF1}(I_p, I_g) \quad (1)$$

$$S_i = \exp\left(-\frac{\mathrm{TED}(A_p, A_g)}{25}\right) \quad (2)$$

$$C_i = \exp\left(-\left|\log \frac{\mathrm{len}(p_i)+50}{\mathrm{len}(g_i)+50}\right|\right) \quad (3)$$

$$Q_i = V_i^{0.85} \cdot S_i^{0.12} \cdot C_i^{0.03} \quad (4)$$

where I_p, I_g render predicted/reference SVGs to 256×256 grayscale images and A_p, A_g extract XML tree structures.

6 Conclusion

6.1 What Worked

1. **Merge-and-retrain** was our single largest improvement (+1.4 Kaggle points). Baking LoRA knowledge into base weights and applying fresh adapters allows progressive capability building without the LoRA rank bottleneck.

2. **Competition-only training data** consistently outperformed mixed datasets. Distribution match matters more than volume for this task.

3. **Greedy decoding** with moderate repetition penalty (1.1) is optimal for structured SVG output. Any sampling or penalty variation degrades performance.

4. **Coordinate rounding** (1 decimal place) provides 54% character savings with negligible visual impact, dramatically improving context efficiency.

6.2 What Didn't Work

1. **External data** from SVGX and OmniSVG degraded performance by 2.23 points despite 3× more samples.

2. **Full fine-tuning** achieved lower loss but worse generation quality than LoRA, likely due to overfitting.

3. **Code generation** via Python API produced shapes too simple for SSIM scoring.

4. **Best-of-N selection** with heuristic scoring (longest valid SVG) does not correlate with visual quality.

5. **Verbose system prompts** hurt rather than help, interfering with learned patterns.

6.3 Future Directions

Several promising directions remain unexplored due to time constraints:

- **DPO with rendered scoring:** Directly optimizing SSIM via Direct Preference Optimization would bypass the CE loss disconnect. Generate SVG pairs, render both, train the model to prefer the higher-SSIM output.

- **Custom coordinate tokenization:** Adding dedicated vocabulary tokens for (x,y) coordinate pairs (the OmniSVG approach) could achieve 3–4× sequence compression.

- **Relative path preprocessing:** Our validated rel+int preprocessing (40% token savings, 0.973 SSIM) was never deployed to the final training pipeline due to time constraints.

- **max_tokens=1536 at submission:** Our ablation shows this is the single easiest improvement (+1.71 points) that we never tried on Kaggle.

6.4 AI Tooling Disclosure

Claude Code (Anthropic) was used extensively throughout this project for code generation, debugging, experiment orchestration, and this report. Specific contributions include: training scripts, inference pipelines, ablation automation, DGX Spark Docker debugging, and figure generation. All AI-generated code was reviewed and understood before use.

6.5 Code and Model Artifacts

All code and model weights are publicly available:

- **GitHub Repository:** https://github.com/ivanearisty/svg-gen — training scripts, inference pipelines, ablation automation, preprocessing, scoring, and this report's LaTeX source.
- **Model Weights (HuggingFace):** https://huggingface.co/kaleidoscopicwhether/svg-gen-weights — all trained models listed below.

Table 6 lists all model artifacts with direct links.

Model	Type	Kaggle
componly-r32-adapter	Adapter	**16.87**
merged-1.5b-r16	Full	(base)
refined-7000	Full	16.26
r16-3epoch	Adapter	15.47
mixed-r32-adapter	Adapter	14.64
codegen-1.5b	Full	12.26

Table 6: Model artifacts on HuggingFace. The best model (componly-r32-adapter) requires merged-1.5b-r16 as its base. All other full models are standalone.

References

Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. 2023. Qlora: Efficient finetuning of quantized language models. *Advances in Neural Information Processing Systems*, 36.

Daniel Han and Michael Han. 2024. Unsloth: Fast and memory-efficient llm fine-tuning. https://github.com/unslothai/unsloth.

Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, and 1 others. 2024. Qwen2.5-coder technical report. *arXiv preprint arXiv:2409.12186*.

OmniSVG Team. 2024. Omnisvg: A unified scalable vector graphics generation model. HuggingFace: https://huggingface.co/datasets/OmniSVG/MMSVG-Icon.

Ximing Xing. 2024. Svgx: A large-scale dataset for svg generation. HuggingFace: https://huggingface.co/datasets/xingxm/SVGX-SFT-1M.

A Reproducibility

A.1 Hardware

- **kepler:** Lenovo ThinkStation P2, RTX 2000 Ada (16GB), Ubuntu 22.04, CUDA 13.0, Driver 580.126.09
- **Thor:** NVIDIA DGX Spark, 128GB unified memory, ARM64/aarch64, CUDA 13.0, Driver 580.00

A.2 Software

- Python 3.10/3.11, PyTorch 2.10/2.11, Transformers 4.x, TRL 0.29+, PEFT, Unsloth
- Random seed: 42 (all experiments)

A.3 Training Time

- LoRA r=16 (Modal A100): ∼10 hours
- LoRA r=32 (kepler RTX 2000): ∼13.6 hours
- Full fine-tune 1.5B (Thor): ∼12 hours
- Ablation study (kepler): 7.25 hours (8 experiments)

A.4 Cost

Modal cloud GPU: ∼$55–75 total. Local machines: free (pre-existing hardware).